

Review Article

A Review of Deep Reinforcement Learning Methods and Military Application Research

Ning Wang , Zhe Li , Xiaolong Liang , Yueqi Hou , and Aiwu Yang 

Air Traffic Control and Navigation College Air Force Engineering University, Xi'an, China

Correspondence should be addressed to Zhe Li; kongyanshi@126.com

Received 10 December 2022; Revised 30 January 2023; Accepted 9 February 2023; Published 10 April 2023

Academic Editor: Ardashir Mohammadzadeh

Copyright © 2023 Ning Wang et al. This is an open access article distributed under the Creative Commons Attribution License, which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited.

In the area of artificial intelligence, deep reinforcement learning has grown in significance. It has accomplished extraordinary feats and offers a fresh approach to previously challenging challenges, such as controlling a robotic arm and discovering game strategies. The two primary categories of deep reinforcement learning methods—deep reinforcement learning based on value function and deep reinforcement learning based on policy gradient—are initially explained in this study. The limitations of current approaches and the difficulties faced by deep reinforcement learning methods in related domains are further sorted out, and then the future application directions of deep reinforcement learning methods in the military sphere are examined. Finally, a growing trend for deep reinforcement learning techniques is anticipated in military applications.

1. Introduction

Deep Learning (DL) and Reinforcement Learning (RL), two significant subfields of machine learning, have made significant strides in recent years in both the development of theoretical frameworks and the study of practical applications. Deep learning's fundamental premise is to use intricate neural network architectures and nonlinear transformation techniques to efficiently extract low-level data features and create significant and challenging-to-acquire abstract features, leading to efficient data analysis. This method has produced excellent results in image detection, voice recognition, natural language processing, and other areas. The fundamental tenet of reinforcement learning is to continuously improve the action plan while accumulating the experience obtained from interactions between the agent and its environment to get the agent to behave as it wants to in its environment. This method has been widely used in manipulator control, analog simulation, and so on.

Deep Reinforcement Learning (DRL) is a general paradigm that combines DL and RL and has achieved success in a variety of complex scenarios, such as game strategy design

[1], automatic driving [2–4], and logistics resource scheduling [5–8]. Several comprehensive reviews about DRL have been published in recent years. For example, Wang et al. [9] summarized some state-of-the-art DRL methods, divided them into three categories: model-free, model-based, and advanced methods, and outlined the challenges that DRL will face in its future development. Vithayathil Varghese and Mahmoud [10] have surveyed the research associated with multitasking within the deep reinforcement arena and presented state-of-the-art approaches by comparing and contrasting recent solutions. Cao et al. [11] analyzed the research progress and results of deep strengthening in the field of electricity and energy, analyzed the improvement of DRL algorithms in related applications, and evaluated the advantages and disadvantages of different algorithms. Arulkumaran et al. [12] analyzed the benefits of introducing deep learning into reinforcement learning and expounded on the development trend of deep reinforcement learning.

Compared to these surveys, we systematically review the mainstream model-free DRL algorithms and expound on the relationships among different algorithms rather than review most DRL algorithms. Based on this, an overview of the potential military application directions of DRL is provided.

To summarize, this paper presents a detailed survey of model-free algorithms with the following contributions:

- (1) We review the current mainstream deep reinforcement learning approaches and present each approach's contributions, strengths, and weaknesses.
- (2) We analyze potential military applications that may use the DRL approach and describe the advantages and disadvantages of the traditional and DRL approaches.
- (3) We summarize the challenges that the DRL approach may face in specific military applications.

2. DRL Based on the Value Function

Generally speaking, deep reinforcement learning methods can be divided into model-free and model-based. Academic research focuses on the deterministic and static environment where the states are mainly discrete and fully observed [9]. Therefore, current academic research and engineering applications mainly focus on model-free methods.

On this basis, DRL algorithms can be divided into value-function-based DRL algorithms and policy-learning-based DRL algorithms. The value function approach guides an intelligent body to complete a decision by evaluating the value of each action when it is in a specific environmental state. In contrast, the policy learning approach directly outputs the action or the probability distribution of the action by constructing a policy deep neural network. Among them, the strategy learning methods are developed based on the value function methods, so the development of these two methods is described and analyzed below.

2.1. Deep Q-Network. The Deep Q-network (DQN) proposed by Mnih et al. [13, 14] is the pioneer of deep reinforcement learning algorithms. DQN constructs a mapping relationship between environmental states and action values by introducing deep neural networks into reinforcement learning. DQN overcomes the shortcomings of the unstable training effect brought about by combining deep learning and reinforcement learning and provides valuable experience for the close combination of deep learning and reinforcement learning.

2.1.1. Basic Model Structure. DQN is a neural network composed of 3 convolutional layers and 2 fully connected layers, which are represented in Figure 1. Convolutional layers are used to extract high-level features in images, and fully connected layers are used to generate complex mappings between image features and action values. Therefore, DQN takes a series of images as input data and outputs a string of real numbers. These real numbers are defined as the value of each action the agent can take.

2.1.2. Algorithm Structure. Labeled data is often a prerequisite for training an effective deep neural network. However, in the reinforcement learning process, labeled data cannot be obtained, so the key is to design a loss function

without labels to complete the learning process of deep neural networks.

DQN adopts the following loss function during training:

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')} [(y - Q(s,a;\theta_i))^2], \quad (1)$$

where y is calculated from another depth neural network called the target network, it can be seen as a label. Correspondingly, the network that requires training is called a value network and $Q(s,a;\theta_i)$ represents the output of the current network and it is the mapping of DQN from the environmental state to different action values. By deriving the parameters of the value network, we can get the gradient needed for the update of the neural network:

$$\nabla_{\theta_i} L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')} [(y - Q(s,a;\theta_i)) \nabla_{\theta_i} Q(s,a;\theta_i)]. \quad (2)$$

Figure 2 represents the algorithm structure of DQN, where the structure of the target value network is identical to that of the value network. During the training process, the target value network periodically replicates the parameters of the value network, which in turn provides the label data required for the training of the value network.

In addition to using a deep neural network to establish a mapping relationship, the DQN algorithm also adopts an empirical playback mechanism, which provides an essential basis for the stable performance of the deep network in reinforcement learning. The data format used by DQN in the training process is called transition, which is expressed as (s_t, a_t, r_t, s_{t+1}) . The DQN stores the transfer samples obtained from the agent's interaction with the environment into the playback memory unit at each time step t . Small batches of transfer samples are randomly selected from the playback memory unit each time during training, and then the parameters are updated using the stochastic gradient descent method. When training deep networks, a too-strong correlation between training data often leads to the unstable performance of neural networks in different scenarios, which is an important reason why deep learning methods and reinforcement learning methods have yet to be well integrated. DQN dramatically reduces the correlation between samples by playing back memory units and random sampling, thus improving the stability of deep reinforcement learning algorithms.

Experimental results show that on most Atari 2600 games, DQN outperforms previous reinforcement learning algorithms and achieves a level comparable to that of professional players. The DQN uses the same network models, parameter settings, and training algorithms when solving various types of deep reinforcement learning tasks based on visual perception, indicating that the approach is highly adaptable and general.

2.1.3. Main Contributions. In summary, DQN has three significant contributions to DRL methods:

- (1) DQN uses an empirical replay pool to reduce the correlation between training data, which provides an essential basis for the algorithm's stability in different scenarios.

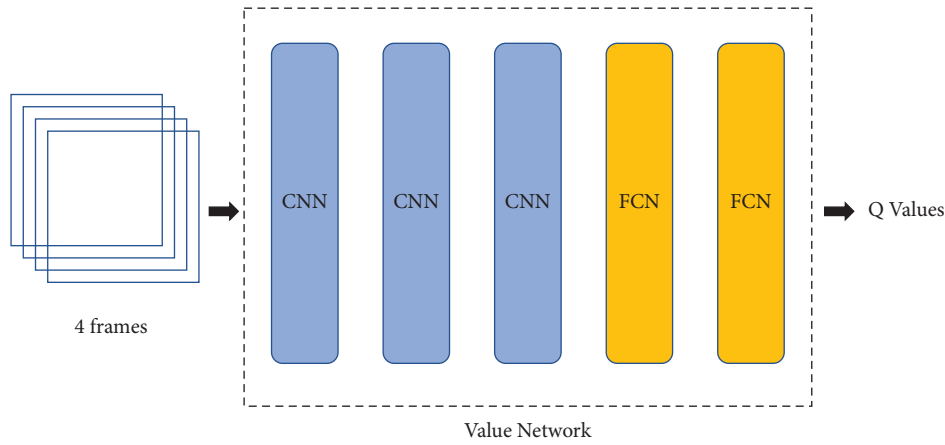


FIGURE 1: Model structure of DQN.

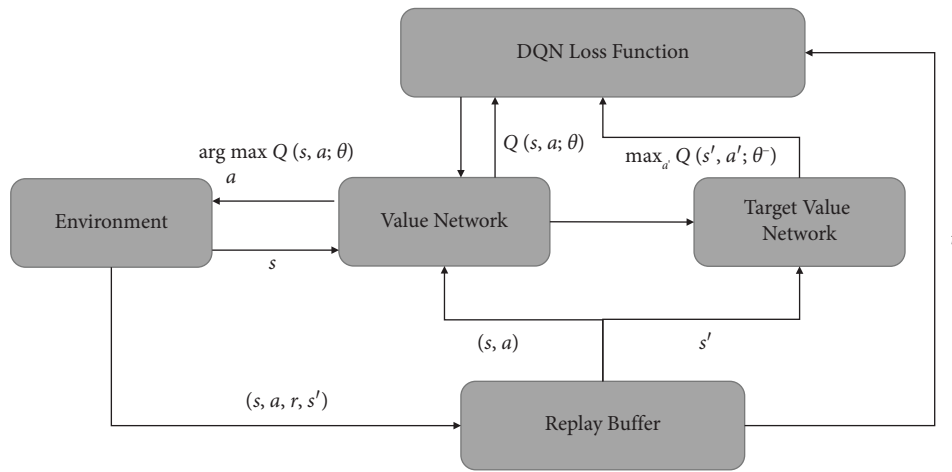


FIGURE 2: The algorithm structure of DQN.

- (2) Deep neural networks are used to achieve the mapping of environmental states to action values so that the algorithm is not limited to a particular scene and thus has a strong generalization performance.
- (3) An end-to-end architecture for deep reinforcement learning methods was established, providing an essential foundation for subsequent research.

The DQN algorithm has shown good performance in some application scenarios, such as video games, but it still has shortcomings. One of the most important is its overestimation problem, where DQN tends to overestimate the Q value of each action, and this overestimation is uneven.

In fact, the performance of deep reinforcement learning algorithms based on value functions depends on the accuracy of Q -value estimation, and the overestimation problem of DQN for Q -value comes from two main aspects. First, in the actual calculation of (1) and (2), DQN replaces the expected value of Q -value by taking the maximum Q -value. Second, the weights of the target value network used to generate the labels are derived from periodically replicating the weights of the value network, and this bootstrapping approach aggravates the overestimation problem of the DQN for Q values.

Based on DQN, a series of improved DQN algorithms have been proposed, most of which focus on the problem of accurate estimation of Q . These algorithms improve the performance of DQN in two main ways: by improving the network structure and by improving the training algorithm.

2.2. Improved DQN Based on Network Structure. Improvements to the DQN model are generally achieved by adding new functional modules to the original network or modifying the original model structure. This section introduces two representative improved DQN algorithms, namely, Dueling DQN [15] and Deep Recurrent Q-Network (DRQN) [16].

2.2.1. Dueling DQN. In the DQN algorithm, the input to the neural network is the state of the environment, and the output is the value of each action. A problem here is that the value of each action contains two parts, one determined by the current environmental state and one determined by the type of action, and when a value is used to reflect the value of these two parts, their evaluations will influence each other. Dueling DQN employs a fully connected neural network

with two branches to reduce the valuation error due to the uncertainty of state value and action value. The structure of the Dueling DQN network is shown in Figure 3.

After the extraction of image features by the convolutional neural network, the two fully connected networks share these image features, which in turn predict the value of the state the agent is in and the advantage value of the action to be taken, respectively, and then finally output them as the Q value. The advantage function is used to represent how beneficial it is to take action over others when in a particular state, and it can be expressed as follows:

$$A(s, a) = Q(s, a) - V(s). \quad (3)$$

Dividing the Q value into two parts for separate estimation helps to improve the accuracy of the Q value. However, this method increases the instability during training due to the increased output variables of the neural network. To solve this problem, during the actual training of Dueling DQN, the Q value of the neural network output is improved as follows:

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right). \quad (4)$$

Calculating the expectation for (4), it can be seen that the second term on the left side is constant to zero, which means that during the training process of the neural network, the change of Q value mainly depends on the change of the environment state, that is, by modifying the formula for calculating Q value, the instability of Q value caused by the change of dominance function is reduced, and thus the stability of the network during training is improved. By decomposing the Q value calculation and designing the Q value substitution formula, Dueling DQN improves the accuracy of the neural network for Q value estimation and the stability during training, which results in better performance.

2.2.2. Deep Recurrent Q-Network. To alleviate the problem of low learning efficiency caused by the partial observability of the agent to the environment state during reinforcement learning, DQN composes the training data from the four most recent frames in which the agent is located (for example, in the Pong game, the neural network is needed to guide the racket to catch the ball. If only one image frame is used as input, it will be difficult for the neural network to extract information about the speed and direction of the ball.), a practice that brings a large computational and storage burden. Deep Recurrent Q-Network (DRQN) [16] effectively alleviates the problem of partial observability of state information by introducing the Long Short-Term Memory (LSTM) neural network structure to memorize the state time information during the interaction between the agent and the environment.

As shown in Figure 4, DRQN replaces the first fully-connected layer in DQN with 256 LSTM. Since LSTM can save and output historical information, the training data of the model at this time can be input only for one frame of the

current moment, reducing the computational resources consumed by the deep neural network during training and use. Experiments show that DRQN exhibits better performance than DQN in the case of partial state observability, and therefore DRQN models are suitable for complex tasks where the problem of partial state observability prevails.

2.3. Improved DQN Based on the Training Algorithm. The overestimation of Q by DQN comes from two sources. One is because it uses the maximum discounted return instead of the discounted return expectation in the Bellman equation:

$$Q^\pi(s_t, a_t) = \mathbb{E}_{r_t, s_{t+1} \sim E} \left[r(s_t, a_t) + \gamma \max_{\pi} [Q^\pi(s_{t+1}, a_{t+1})] \right]. \quad (5)$$

And the other is because it uses its own estimated Q values at two time points for the time difference calculation, which aggravates its overestimation.

Some valuable algorithms have been proposed to solve the overestimation problem in DQN. The Double DQN proposed by van Hasselt et al. [17] is a representative work among them.

In addition, there are other representative works that also improve the performance of DQN and are borrowed by many algorithms developed subsequently, such as prioritized experience replay (PER) [18], multistep bootstrapping target learning, normalized advantage function (NAF) [19], noisy DQN [20], categorical DQN [21], etc. We focus on the preferential experience playback method, which is more general compared to other works.

2.3.1. Double DQN. In the standard DQN algorithm, the update formula of the neural network can be expressed by the following equation:

$$\theta_{t+1} = \theta_t + \alpha (y_t^{\text{DQN}} - Q(s_t, a_t; \theta_t)) \nabla_{\theta_t} Q(s_t, a_t; \theta_t), \quad (6)$$

where the objective value of the optimization is the following equation:

$$y_t^{\text{DQN}} = r_{t+1} + \gamma \max Q(s_{t+1}, a; \theta_t). \quad (7)$$

The abovementioned equation can also be further written as the following equation:

$$y_t^{\text{DQN}} = r_{t+1} + \gamma Q(s_{t+1}, \arg\max Q(s_{t+1}, a; \theta_t); \theta_t). \quad (8)$$

From the above equation, it can be seen that in the DQN algorithm, the selection and evaluation of the actions are done based on the neural network with parameters θ , which can aggravate the problem of overestimation of Q values by the DQN. The Double DQN (DDQN) algorithm proposed by Hasselt et al. based on DQN and Double Q-learning algorithms, alleviates the problem of overestimation of Q values by DQN.

The idea of DDQN is like this, decoupling the value network and the target network in DQN, estimating the policy, and selecting the action through the value network. Then the Q -value of that action is estimated through the target network. The two networks use different parameters so that the Q overestimation problem caused by bootstrapping can be mitigated.

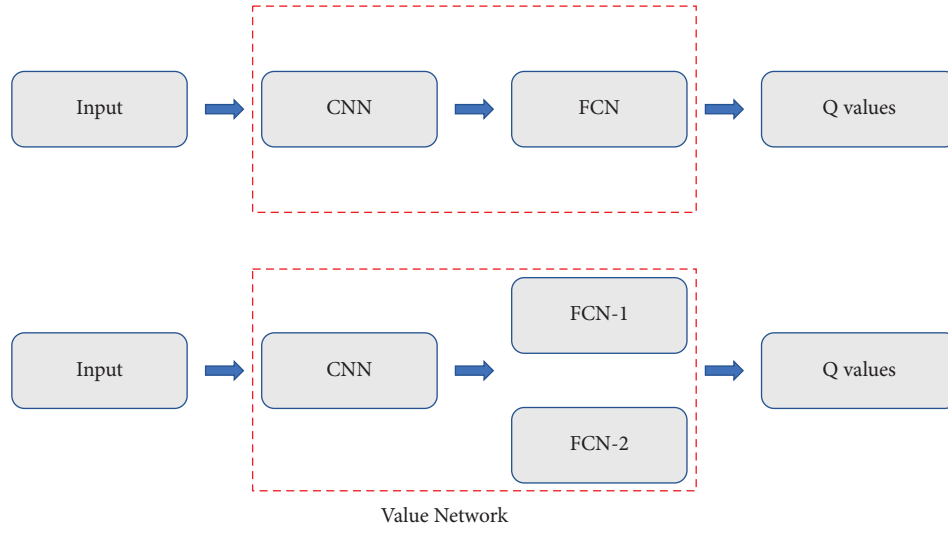


FIGURE 3: Model structure of dueling DQN.

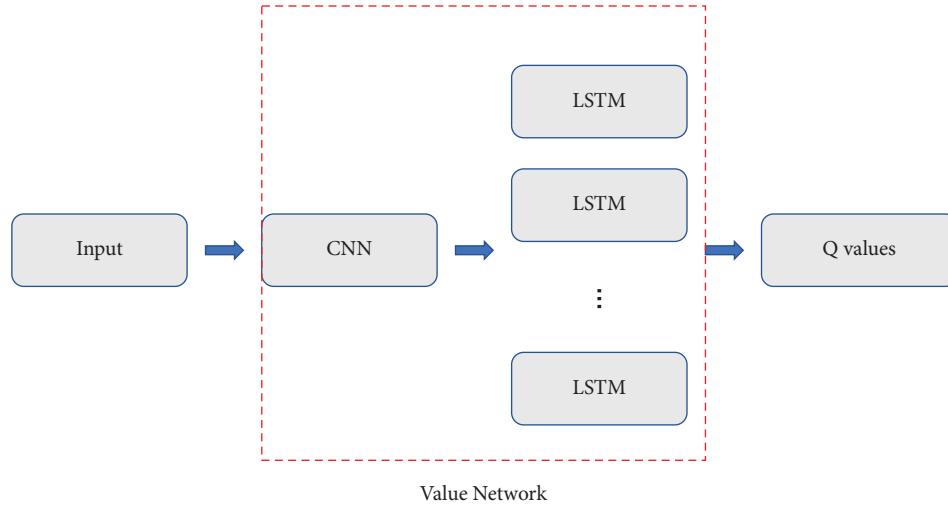


FIGURE 4: Model structure of DRQN.

The algorithm update method of DDQN is the same as DQN, only the generation of the optimized target value is changed:

$$y_t^{\text{DoubleDQN}} = r_{t+1} + \gamma Q(s_{t+1}, \arg\max Q(s_{t+1}, a; \theta_t^-); \theta_t^-), \quad (9)$$

where θ_t is the parameter of the value network and θ_t^- is the parameter of the target network. By separating the processes of action selection and policy evaluation, DDQN alleviates the Q overestimation problem of DQN. Experiments show that DDQN is able to estimating Q values more accurately, which can lead to more stable and effective strategies in some Atari 2600 games.

In fact, setting the parameters of the value network used to generate the policy and the target network used to evaluate the policy separately has many benefits, which have informed later DRL algorithms based on policy gradients.

2.3.2. Prioritized Experience Replay. DQN uses an empirical replay mechanism to effectively eliminate the correlation between training sample data. During the training process, DQN draws small batches of samples with medium probability from the sample pool at each moment for training. However, the amount of learning information that different samples can bring to the DQN is different, which is not noticed by the equal-probability sampling method used by the DQN. In addition, since the storage capacity of the sample pool is limited, the update of the sample pool always uses new samples instead of old ones, resulting in some important training data in the old samples being discarded before they are fully utilized.

To address this problem, Schual et al. [18] proposed a new training data sampling method named prioritized experience replay (PER). This method effectively speeds up the convergence of DQN by increasing the sampling probability of valuable samples during the training of DQN.

PER uses the time difference error term of each sample as a criterion for evaluating the priority. This error term can be expressed by the following equation:

$$TD_{\text{error}} = r + \gamma \max_{a'} Q(s', a' | \theta^-) - Q(s, a | \theta). \quad (10)$$

The corresponding error is calculated for each sample of data according to the above equation, and the larger the absolute value of the error, the higher the probability of the sample being taken. The probability of each transition being extracted can be expressed by the following equation:

$$P(i) = \frac{p_i}{\sum p_i}, \quad (11)$$

where the p_i can be replaced by the TD_{error} calculated from the i -th transition. In this way, the probability of large error samples being used for training can be effectively increased, which can accelerate the convergence of the network.

However, assigning a higher probability of being sampled to samples with large errors poses two problems. On the one hand, this approach reduces the diversity of training samples while accelerating model convergence, which can make it difficult for the neural network to learn the parameters with good enough generalization. On the other hand, the introduction of sample prioritization accelerates the convergence of the network but also leads to changes in the probability distribution of the samples, which can cause the neural network to converge to different results.

Therefore, the method also employs two techniques, stochastic prioritization and importance sampling weights, in the sampling of the training data. Among them, stochastic prioritization not only makes full use of the samples corresponding to larger TD error terms but also ensures the diversity of the samples drawn. With the stochastic prioritization, p_i can be calculated by

$$p_i = |TD_{\text{error}} + \epsilon|, \quad (12)$$

where epsilon is a very small value used to prevent the probability of a particular transition being drawn to be 0. It is guaranteed that every transition in the experience pool has the possibility of being used for training. It can be seen that by changing the calculation method of p_i , the diversity of training data is improved and thus the risk of overfitting the neural network is reduced.

The importance sampling method attenuates the effect of changes in the sample probability distribution, it updates the loss function of the neural network:

$$\theta_{t+1} = \theta_t + \alpha \cdot \frac{1}{m} \sum_{i=1}^N \omega_i (y_t^i - Q(s_t, a_t; \theta_t)) \nabla_{\theta_t} Q(s_t, a_t; \theta_t). \quad (13)$$

The ω_i in (13) can be calculated by

$$\omega_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta, \quad (14)$$

where β is a hyperparameter between 0 and 1 to indicate the extent to which the effect of changes in the probability distribution on the training results is offset. When β is taken

to be 1, it means that the effect of the change in probability distribution brought about by using a priority ranking is completely offset, which also means that there is no difference between the PER and the empirical playback in the standard DQN.

Experiments show that PER can effectively improve the training efficiency of the off-policy algorithm and enable the algorithm to achieve better performance in scenarios, such as Atari games.

3. DRL Based on the Policy Gradient

The DRL method based on value functions has the advantage of a simple network structure, but the method is oriented towards control problems where the control volume is a discrete finite space. The action space of many control problems is continuous, an effective way to apply a value function-based DRL method like DQN to the constant domain is to discretize the action space of the control problem. But with this approach, the action space grows exponentially with the increase of control degrees of freedom, which poses significant difficulties for training. The DRL method based on policy gradients proposes a better solution for the control problem in the continuous domain.

DRL methods based on policy gradients can be divided into three main categories: Deep Deterministic Policy Gradient (DDPG) [22], Trust Region Policy Optimization (TRPO) [23], and Asynchronous Advantage Actor-Critic (A3C) [24], based on which a series of improved algorithms have been derived.

3.1. Actor-Critic. In the value function-based deep reinforcement learning approach, the neural network output is the evaluation of each action taken by an agent in a specific environmental state. The policy gradient-based deep reinforcement learning approach takes another idea, which makes the neural network output the optimal probability distribution of an action taken by the agent in a specific environmental state and then randomly samples this probability distribution to finally obtain a definite action. A neural network with an output action probability distribution is called a policy network, and the gradient to be used for updating such a network is called the policy gradient. Among them, AC is a typical deep reinforcement learning method based on policy gradients, and many subsequent algorithms have been developed based on the AC algorithm.

The AC network structure consists of two parts: the actor network and the critic network, as shown in Figure 5. Actor is a policy network, which is used to output the action probability distribution π when the agent makes a decision, and its optimization objective can be expressed by the progress towards maximizing the following equation:

$$J(\theta) = \mathbb{E}_S [V_\pi(S)]. \quad (15)$$

In (15), $V_\pi(S)$ denotes the state value function, which can be defined by the following equation:

$$V_\pi(s_t) = \mathbb{E}_{A_t \sim \pi(\cdot | s_t; \theta)} [Q_\pi(s_t, A_t)]. \quad (16)$$

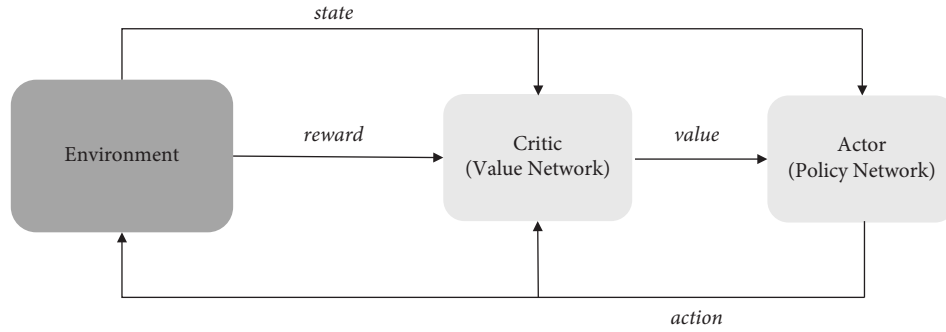


FIGURE 5: Model structure of AC.

The gradient of (15) was derived by Sutton et al. [25] and the following results were obtained:

$$\frac{\partial J(\theta)}{\partial \theta} = \mathbb{E}_S \left[\mathbb{E}_{A \sim \pi(\cdot|S;\theta)} \left[\frac{\partial \ln \pi(A|S;\theta)}{\partial \theta} \cdot Q_\pi(S, A) \right] \right]. \quad (17)$$

The (17) is called the Policy Gradient (PG), which is used to update Actor. However, when calculating the PG, the uncertainty of the action value function $Q_\pi(S, A)$ needs to be removed by finding the expectation. And in practice, we can only find the approximate expectation, so the resulting strategy gradient is also called the approximate strategy gradient.

The Monte Carlo approximation is used in the AC algorithm to find the approximate policy gradient. The corresponding expected values $E_S(\cdot)$ and $E_A(\cdot)$ needed for the calculation are replaced by using the observed state s and the actual adopted action a :

$$g_{PG}(s, a; \theta) \triangleq Q_\pi(s, a) \cdot \nabla_\theta \ln \pi(a|s; \theta). \quad (18)$$

In the process of calculating the policy gradient, the $Q_\pi(s, a)$ is calculated using the value function method, i.e., a neural network is used to estimate the $Q_\pi(s, a)$, which in the AC algorithm is called the criterion. Unlike DQN, the input of DQN is the current environment state of the agent, and the output is the Q -value of each action in the action space. The input of critic is the current environment state of the agent and the action taken by the agent, and the output is the Q -value of the specific action used to calculate the policy gradient. The update of the critic neural network uses the TD error-based gradient descent method, while the update of the actor neural network uses the gradient ascent method based on the policy gradient.

3.2. Deep Deterministic Policy Gradient. For the AC algorithm, the actor outputs a probability distribution of the action space, and the actual action taken by the agent needs to be obtained by random sampling. Since the uncertainty of the actions further leads to the uncertainty of the policy gradient during training, this method is called stochastic policy gradient. The stochastic policy gradient can effectively explore the solution space to discover a better decision method but it still has some problems. One of the more prominent ones is that the stochastic policy gradient method

is an on-policy method, which makes it difficult to reuse the training data obtained during the interaction between the agent and the environment. This also leads to the inefficient utilization of training samples by the AC algorithm.

Silver et al. [26] gave a method to calculate the deterministic policy gradient (DPG) as in (19), and this method solves the problem of low utilization of samples by the AC algorithm from another perspective.

$$\frac{\partial J(\theta)}{\partial \theta} = \mathbb{E}_S \left[\frac{\partial \mu(S; \theta)}{\partial \theta} \cdot Q_\mu(S, a) \Big|_{a=\mu(S; \theta)} \right]. \quad (19)$$

Lillicrap et al. [22] introduced the idea of DQN based on the DPG and AC algorithms and proposed the more practical DDPG algorithm. The network structure of DDPG is shown in Figure 6 DDPG outputs a definite action value instead of an action probability distribution, and a definite action can bring a definite policy gradient, so DDPG becomes an off-policy algorithm based on policy gradient. The off-policy algorithm can reuse the training sample data obtained from the interaction between the agent and the environment and can use techniques such as PER to improve the efficiency of neural network training, so DDPG outperforms the AC algorithm and its other improved versions in terms of sample utilization.

DDPG uses the same empirical replay mechanism and separate target network as DQN and is based on the AC algorithm to reduce data correlation and increase the stability and robustness of the algorithm. In addition, DDPG uses a soft update method to update the target network. The target network of DQN is synchronized with the Q network every N steps, while the respective target network parameters θ^- and ω^- of actor and critic in DDPG are updated by changing more slowly instead of directly copying the parameters, thus further increasing the stability of the learning process:

$$\begin{aligned} \theta^- &= \tau \theta + (1 - \tau) \theta^-, \\ \omega^- &= \tau \omega + (1 - \tau) \omega^-. \end{aligned} \quad (20)$$

Adopting a deterministic strategy enables DDPG to utilize the training samples efficiently. However, it also brings some disadvantages, the most prominent of which is that the limitation of the deterministic strategy reduces the ability of DDPG to explore the decision space. To alleviate

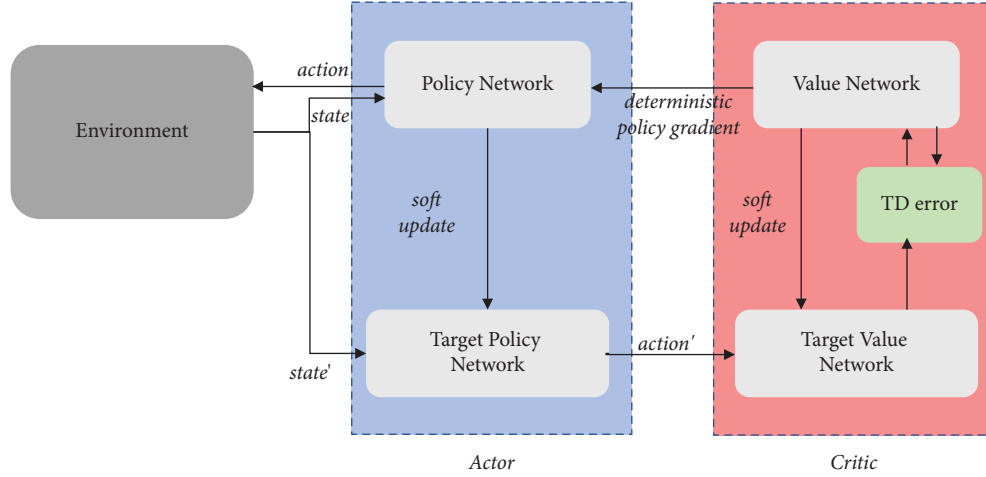


FIGURE 6: Model structure of DDPG.

this problem, DDPG usually adds a noise term N when generating strategies μ' to increase the network's ability to explore the decision space and increase its robustness:

$$\mu'(s_t) = \mu_s(s_t) + N. \quad (21)$$

Experiments show that DDPG can achieve better results than DQN in Atari games and can be applied to control problems with continuous motion space, such as robotic arm control. One obvious advantage of DDPG is its simplicity. However, the Critic network in DDPG uses the same TD error update method as DDQN, so Critic suffers from Q value overestimation bias and bias accumulation problems. This bias has a negative impact on the strategy learning of the Actor.

3.3. Twin Delayed DDPG. Fujimoto et al. [27] made valuable improvements to DDPG and proposed the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm. The upgrades to the TD3 algorithm include three main aspects, which are described separately below.

For the problem of Critic's overestimation of Q values in DDPG, TD3 modifies the network structure of DDPG, as shown in Figure 7.

For the problem of Q value overestimation in DDPG, firstly, TD3 uses two networks to find the Q-value separately, and since the parameters of the two networks are randomly initialized, the output Q-values must be large or small. They are counted as Q1 and Q2, respectively, and the smaller value is taken as the Q-value value, so there is a new way of calculating the policy gradient:

$$\frac{\partial J(\theta)}{\partial \theta} = E_S \left[\frac{\partial \mu(S; \theta)}{\partial \theta} \cdot \min_{i=1,2} [Q_\mu^1(S, a), Q_\mu^2(S, a)|_{a=\mu(S; \theta)}] \right]. \quad (22)$$

For Critic, both value networks use the smaller Q value of the two target network outputs to calculate the TD error and perform parameter updates by gradient descent. By adding two networks with different initialization parameters, TD3 alleviates the problem of overestimation of Q values by Critic in DDPG and thus achieves better performance.

The second improvement of TD3 is the proposed delayed policy update method, which is based on the idea that the update frequency of Critic is tuned a little higher than that by Actor to avoid blind updates of Actor and thus improve the training effect.

The third improvement of TD3 is target policy smoothing. In DDPG, the Q-target is calculated by inputting state and action to obtain the Q-target and estimate the Q-target value of the target state-action pair s' and a' ; while in TD3, the state and action after adding noise are input so that the action is randomized within a specific range (the value is limited to $a_{\text{low}} \leq a \leq a_{\text{high}}$), thus making the target network more robust in estimating the Q-value and thus improving the stability of the network during training.

Overall, the TD3 algorithm better mitigates the value overestimation problem in the DDPG algorithm by adding a new pair of Critic networks and performs better than DDPG. In the test results of all tasks in openAI's gym, the TD3 algorithm performs better than DDPG, PPO, TRPO, and other algorithms.

3.4. Proximal Policy Optimization. Proximal policy optimization (PPO) is another family of improved algorithms for the AC algorithm. It is developed based on the Trust Region Policy Optimization (TRPO) algorithm. Compared with the TRPO algorithm, the PPO algorithm is more concise and more effective [28]. TRPO is an algorithm proposed by Schulman et al. [23] in 2015, and it is a successful application of trust-region methods in DRL. TRPO uses the following objective function for the optimization of the actor:

$$\text{maximize } \hat{\mathbb{E}}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t \right], \quad (23)$$

$$\text{subject to } \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]] \leq \delta, \quad (24)$$

where $\hat{A}_t$ in (23) denotes the advantage function, which can be calculated from equation (26). The Advantage Actor-Critic (A2C) algorithm has demonstrated that using the

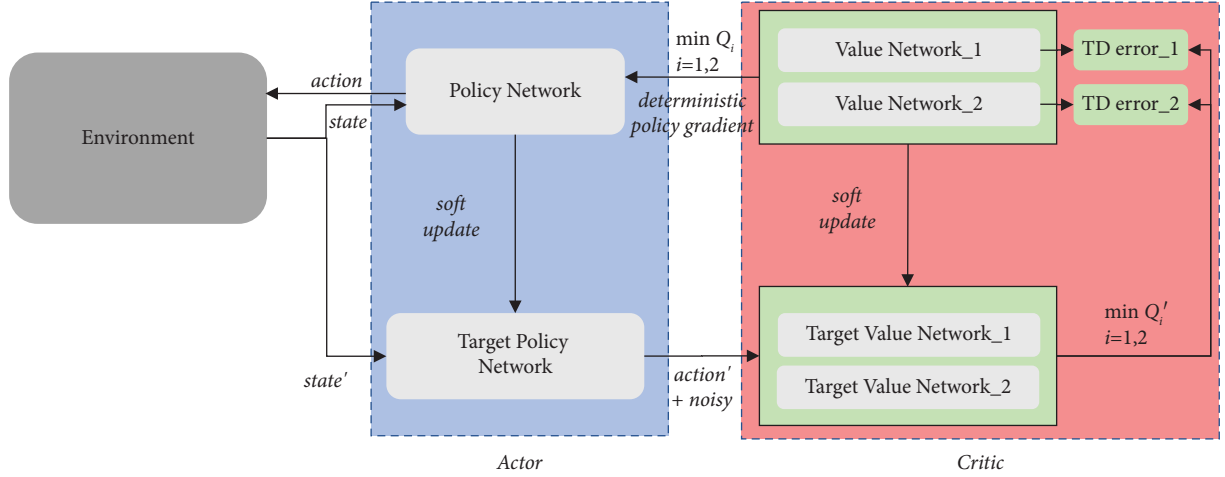


FIGURE 7: Model structure of TD3.

advantage function instead of the Q value when calculating the policy gradient can lead to a smaller gradient variance and thus improve the network's training:

$$\hat{A}_t = Q_\pi(s, a) - V_\pi(s). \quad (25)$$

In particular, (24) is used to limit the magnitude of each parameter update during TRPO training. By limiting the magnitude of each parameter update during the neural network training, TRPO achieves more stable performance. However, the method of iteratively updating the algorithm by constructing an approximation function in TRPO is more complicated, based on which PPO adopts a straightforward and more effective approach. The following equation can express the objective function used by the PPO to update the actor:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_t \right) \right], \quad (26)$$

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}. \quad (27)$$

The basic idea of PPO is to represent the restriction $r_t(\theta)$ in TRPO on the amplitude of strategy updates through a truncated function, i.e., the combination of (23) and (24), expressed as (26), to simplify the calculation process of KL scatter and improve the computational efficiency. The basic content of (26) can be represented by Figure 8, and its basic meaning can be understood as follows:

- (1) If $A > 0$, the strategy at this stage is relatively good. That is, (s_t, a_t) is relatively good. As the optimization goal of the strategy function is to maximize $r_t(\theta) \hat{A}_t$, the higher the probability of occurrence of the binary, the better, that is, the larger the value of $r_t(\theta)$ the better, but to ensure the stability of the strategy update, the value of the denominator and numerator in $r_t(\theta)$ can not differ too much, so it is necessary to add an upper limit.
- (2) If $A < 0$, the strategy at this stage is relatively poor, i.e., (s_t, a_t) is rather poor. Since the optimization goal

of the strategy function is to maximize $r_t(\theta) \hat{A}_t$, the smaller the probability of the binary occurring, the better, that is, the lower the value of $r_t(\theta)$, but to ensure the stability of the strategy update, the value of the denominator and numerator in $r_t(\theta)$ can not differ too much, so a lower limit needs to be added.

PPO simplifies the problem of solving the objective function with constraints compared to TRPO's problem of solving the objective function without constraints and has more stringent restrictions on parameter updates. Experiments show that the PPO algorithm has the same stability and reliability as TRPO and is more concise.

3.5. Soft Actor-Critic. The current advanced on-policy deep reinforcement learning methods, such as TRPO, PPO, A3C [24], etc., have an obvious problem that the utilization of training samples is very inefficient, which in turn leads to very high training costs and is difficult to be practically applied in realistic scenarios. Although the DDPG and TD3 algorithms, which are improved based on DDPG, are off-policy algorithms and can reuse training data, they have the problem of being too sensitive to hyperparameters and require much effort to adjust hyperparameters to achieve the desired training effect, which makes them difficult apply to complex problems.

Soft Actor-Critic (SAC) [29] well resolves the paradoxical relationship between algorithm stability and inefficient utilization of training samples. The state value function and action value function of the maximum entropy-based reinforcement learning framework used by SAC can be expressed by the following equation:

$$\begin{aligned} V(s_t) &= \mathbb{E}_{a_t \sim \pi} [Q(s_t, a_t) - \alpha \log \pi(a_t|s_t)], \\ Q(s_t, a_t) &= r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V(s_{t+1})]. \end{aligned} \quad (28)$$

The Q value under any policy can be calculated by iterative computation based on the above equation. The following equation can give the objective function and gradient of the actor and critic:

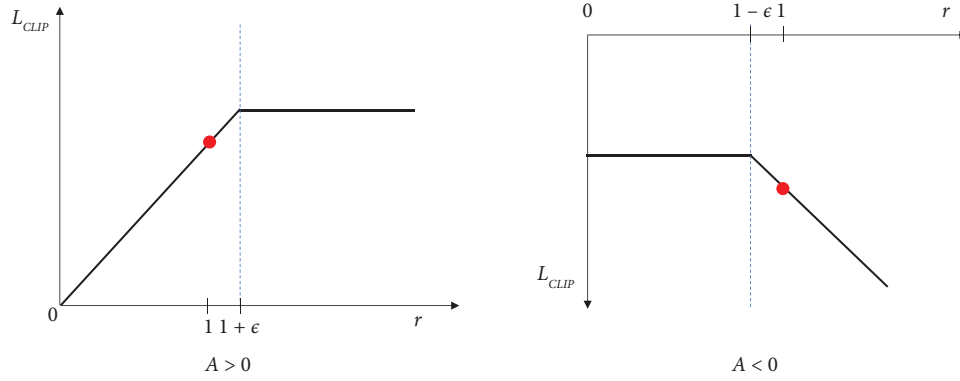


FIGURE 8: Schematic of the truncation function in PPO.

$$J_Q(\theta) = \mathbb{E}_{(s_t, a_t) \sim D} \left[\frac{1}{2} \left(Q_\theta(s_t, a_t) - (r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p} [V_{\bar{\theta}}(s_{t+1})]) \right)^2 \right],$$

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim D} \left[\mathbb{E}_{a_t \sim \pi_\phi} \left[\alpha \log(\pi_\phi(a_t | s_t)) - Q_\theta(s_t, a_t) \right] \right]$$
(29)

SAC improves the algorithm's ability to explore the decision space by adding an entropy term to the objective function and enhances the algorithm's robustness by adding noise to the action sampling, which enables the algorithm to show excellent performance in many scenarios.

3.6. Future Research Directions. The rapid development of deep reinforcement learning provides new solutions to problems such as automatic control and intelligent decision-making, but at the same time, there are still problems that need to be solved, such as low learning efficiency and single adaptation scenarios. Combined with the existing research results, the following technologies will become essential research directions in the field of deep reinforcement learning in the future.

3.6.1. Hierarchical Deep Reinforcement Learning. In some complex DRL tasks, optimizing strategies directly with the final goal in mind is inefficient. Therefore, hierarchical reinforcement learning (HRL) can be used to decompose the final goal into multiple subtasks to learn hierarchical strategies and to form an effective global strategy by combining the strategies of multiple subtasks [30]. Therefore, with the increasing complexity of intelligent decision problems, it will become a very challenging topic to solve the reinforcement learning problem in sparse reward problems by hierarchically designing reward functions.

3.6.2. MultiAgent Deep Reinforcement Learning. A single-agent system's decision-making capability is insufficient when faced with complex decision-making problems in some real-world scenarios [30, 31]. For example, in an Atari 2600 game with multiple players, multiple decision-makers are required to cooperate or compete with each other. Therefore, in specific situations, the DRL model needs to be

extended to a multi-agent system with multiple agents cooperating, communicating, and competing with each other. Efficient competition and cooperation mechanisms between multiagent systems will be an essential way to take advantage of deep reinforcement learning of multiagent systems.

3.6.3. Transfer Deep Reinforcement Learning. One limitation of DRL is that it does not learn as efficiently as humans do, and a DRL model that can be practical often requires a large number of computational resources for training. Transferring learning is a potential approach that may solve this problem. Current research on transfer DRL can be divided into three main categories: (1) forward transfer that trains on one task and transfers to a new task; (2) multitask transfer that trains on many tasks and transfers to a new task; and (3) meta-RL that learns to learn from many tasks [9, 32]. How to choose the appropriate transfer learning method according to specific problems will become an important research area to reduce the computational cost of DRL algorithms and expand the application scope Table 1.

4. Military Applied Research on Deep Reinforcement Learning

DRL brings new tools to many military decision and control problems that have not been well addressed. This section provides an analytical overview of the application of deep reinforcement learning in military scenarios, focusing on three aspects: air combat maneuvering decisions, task assignment, and military chess deduction. Deep reinforcement learning has great potential for application in these three directions, so we do not limit ourselves to deep reinforcement learning methods but also introduce existing, more mature methods to provide readers with a more comprehensive understanding of the related fields.

TABLE 1: Contributions, advantages, and disadvantages of mainstream deep reinforcement learning methods.

DQN	[13]	An empirical playback mechanism and a time-series differential error loss function are used to propose a practical deep reinforcement learning algorithm	Inaccurate estimation of action value and serious overestimation problems
Dueling DQN	[15]	Reduced coupling between environmental state values and action values results in more accurate estimation of action values	There is still a problem of overestimation of action value and it cannot be applied to the control problem of continuous space
Double DQN	[17]	Separate action selection and value estimation to avoid overestimation of value	There is still a problem of overestimation of action value and it cannot be applied to the control problem of continuous space
Rainbow DQN	[33]	Integrating the advantages of a series of improved versions of the DQN algorithm, a DQN algorithm with more superior performance is proposed	The algorithm is still oriented to the discrete decision space, and it is difficult to effectively solve the control and decision problems in continuous space
A3C	[24]	Uses multiple threads to interact with the environment to improve the speed of algorithm training	Typically converge to a local rather than global optimum and evaluating a policy is typically inefficient and high variance
TRPO	[23]	A stable deep reinforcement learning algorithm that can guarantee the monotonic increment of the policy optimization process is proposed	Algorithm theory is too complex and not easily deployed
PPO	[28]	Simplifies the complexity of the algorithm while achieving essentially the same performance as TRPO, and uses importance sampling to improve the training efficiency of the algorithm	It is still an on policy algorithm, which is not efficient in using training data and expensive in practical application
DDPG	[22]	Combining the advantages of DQN and DPG, an off-policy deep reinforcement learning algorithm for the continuous domain is proposed	The algorithm is very highly tuned to hyperparameters and is not very exploratory of the decision space
TD3	[27]	Takes the minimum value between a pair of critics to restrict overestimation and delays policy updates to reduce per-update error	Algorithm performance is still too sensitive to hyperparameters
SAC	[29]	The use of an objective function that includes an entropy term enhances the exploratory and stability of the algorithm, which is an off-policy algorithm that makes full use of the training data and reduces the cost in practical application scenarios	Algorithm performance is more sensitive to the setting of temperature parameters, and the design of adaptive temperature parameters is complicated

4.1. Air Combat Maneuvering Decision

4.1.1. Mission Characteristics Analysis. Autonomous air combat maneuver decision-making refers to the process of automatically generating the maneuver control decisions of the fighter based on mathematical optimization and artificial intelligence [33], which requires that UAVs have independent capabilities for autonomous sensing, information processing, and decision-making abilities [34]. In August 2020, in the AlphaDogfight competition organized by the Defense Advanced Research Projects Agency (DARPA), Heron Systems defeated the U.S. Air Force F-16 human pilot with a record of 5:0 and won the championship, breaking the U.S. military's perception of traditional air combat. Although the U.S. military has not disclosed details of the technology, autonomous air warfare technology based on DRL is beginning to receive widespread attention.

Modern air warfare can be divided into two aspects: within visual range (WVR) air combat and beyond visual range (BVR) air combat [35]. WVR air combat mainly considers the relative position and maneuvering performance of both aircraft and creates an altitude, angle, and speed advantage over enemy aircraft through maneuvering decisions, thus increasing the probability of winning in air combat. The problem of autonomous decision-making in BVR air combat and WVR air combat is somewhat different, specifically in two ways. One is that medium- and long-range air-to-air missiles are mainly used for BVR. The tactical mechanism and rules of the missile are different from short-range air combat. Largely, no mature theory has yet been formed to guide pilots. Second, the two sides cannot use a real missile for training. It is difficult to calculate the damage assessment of the missile when one side makes a maneuvering action [36]. Therefore, when conducting BVR air combat, the state space and maneuver space of the aircraft become larger, and it becomes very difficult to deduce the maneuver intention of the enemy aircraft, thus increasing the difficulty of maneuver decision. More research is related to WVR air combat maneuvering decisions, and some more mature autonomous decision methods have been developed, some of which are still valuable for BVR air combat.

4.1.2. Advantages and Disadvantages of Existing Methods. The tactical maneuver decision based on an analytical solution converts the tactical maneuver decision problem into a mathematical optimization problem and selects an optimization objective that meets the characteristics of air combat to achieve the maneuver decision, and the analytical solution method covers optimal control theory [37, 38], objective optimization theory [39, 40], etc. Li et al. [33] proposed a rolling time-domain control-based air combat maneuvering decision method, and different indicator functions were constructed in the study to enable effective decision-making in threat avoidance situations; Park [41] combined matrix countermeasures with intelligent algorithms, and the idea was to perform an approximate optimal solution based on matrix countermeasures as well as genetic

algorithms to improve decision accuracy, which was verified by simulation to be reasonable, but there is the problem that the real-time performance is difficult to meet the demand of close air combat decision making. In general, the mathematical-based solution method is clear and easy to understand in its mathematical expression, but there are often difficulties in solving it is difficult to find the exact solution, and it is difficult to apply it for maneuver decisions in a complex environment.

Air combat maneuver decision based on an intelligent optimization algorithm on the basis of establishing maneuver decision instruction solution space, based on the objective dominance function using an intelligent optimization algorithm for global traversal, from which the optimal solution vector is searched as the solution scheme of maneuver decision, machine search methods cover Monte Carlo search [42], an intelligent algorithm [43], etc. Among them, Lin [44] proposed a maneuver decision method based on hybrid algorithm, which combined with potential field theory to search for the best maneuver decision direction; In general, the research based on machine search methods is divided into two aspects: first, the design of the objective function optimized to fit the actual air combat characteristics; and second, the improvement of the search algorithm to improve the real-time and accuracy of the solution of the decision scheme. The intelligent optimization method is currently the most commonly used method and has been validated at the practical application level, but further research is still needed for optimization objectives and real-time performance.

4.1.3. Advantages and Challenges of Deep Reinforcement Learning Methods. In the air combat decision-making process, both the analytical solution method and the intelligent optimization method face certain limitations. Among them, the analytical solution method requires accurate and description of the decision model and cannot be applied to air warfare scenarios without models. The disadvantage of the intelligent optimization method is that it is difficult to establish a suitable solution space, and the related research is usually conducted in a two-dimensional plane [45], which is difficult to directly apply to the actual air combat game process.

Deep reinforcement learning methods use a "trial-and-error" approach to interact with the environment and improve its decision-making through continuous learning experiences as the intelligence interacts with the environment. It overcomes the disadvantages of other methods, such as complex modeling, difficult sample labeling, and tedious solving, and generates a series of air combat decision sequences with long-term effects through self-interaction training without human intervention. Some scholars have noticed the potential of deep reinforcement learning for application in air combat decision-making and have conducted some related research. Zhang et al. [46] propose a heuristic Q-Network method that integrates expert experience and uses expert experience as a heuristic signal to guide the search process. Based on this, Zhong et al. [47]

used visual sensor information as input data and a DQN network to achieve autonomous decision-making for air combat maneuvers in 3D space. Bo et al. [48] used the more advanced TD3 algorithm as a decision network and obtained better decision results.

Although deep reinforcement learning methods have been used to study the autonomous air combat decision-making problem, most are focused on the single aircraft proximity air combat decision-making problem under simple scenario strips, and deep reinforcement learning methods in autonomous air combat decision-making still face the following challenges.

- (1) A practical reward function is needed to guide the intelligence to keep learning, and in air combat decision-making tasks, the rewards are usually sparse, and sparse rewards make neural network training less efficient.
- (2) The level of air combat decision-making is often strongly related to the level of adversary decision-making, and establishing an adversary with a high level of decision-making is usually a challenging problem.
- (3) The actual air combat is usually played by a long wing formation of 2 fighters, and the air combat decision of the formation requires complex cooperative tactics, and the air combat decision of a single aircraft usually does not have broad applicability.

4.2. Task Assignment

4.2.1. Mission Characteristics Analysis. Mission allocation is a critical issue to be studied in current operations, and its purpose is to assign each combat mission to the correct elements to perform to maximize the effectiveness ratio of combat resources. This issue is significant in air defense operations and cluster operations.

Currently, algorithms for solving task assignments can be broadly classified into three categories: analytical solution methods, contract network-based task assignment methods, and heuristic intelligent optimization algorithms.

4.2.2. Advantages and Disadvantages of Existing Methods. As a deterministic algorithm, the analytical solution method can give an optimal solution to the task assignment problem. However, the difficulty of modeling and solving the problem increases fast when solving a large-scale assignment problem, which leads to two issues. First, for dynamically changing tasks, the analytical solution method needs to build a dynamically changing task assignment model, which usually not fast to build in practical applications. Second, the analytical solution method requires many computational resources to solve large-scale task assignment problems. Otherwise, the required results cannot be achieved in the given time.

The contract network-based approach treats task assignment as a negotiation process in which the manager issues a task notice and task performers compete to obtain

the right to perform the task in the form of a bid. This approach is very flexible and can effectively solve task assignment problems in scenarios, such as dynamic changes in organizational size but it also has some limitations. First, it is difficult to consider the interrelationship between different tasks in the contract network protocol approach, so this approach is usually only applicable to task assignments of multiple unrelated tasks. Second, the contract network protocol requires repeated negotiation between managers and task executors, which consumes many communication resources. Moreover, when the manager establishes an agreement with an executor, other executors may still bid to the manager, which will bring unnecessary extra computation.

Intelligent optimization algorithms, represented by genetic algorithms and ant colony algorithms, have attracted the research interest of many scholars in task assignments because they do not require the continuity or derivability of the objective function. Intelligent optimization algorithms have apparent advantages in solving complex task assignment problems and can obtain satisfactory solutions faster. However, the intelligent optimization algorithm has some shortcomings in the practical application process. One is that the algorithm is prone to fall into local optimality, which leads to missing the optimal task assignment solution. Second, the parameter setting of the algorithm often has a significant impact on the effect of the algorithm, and the parameter setting is usually done by relying on experience.

4.2.3. Advantages and Challenges of Deep Reinforcement Learning Methods. Compared with other task assignment methods, deep reinforcement learning-based task assignment methods are usually built on multiintelligent systems, and this approach has the following advantages in dealing with the highly game-like and complex military task assignment problem.

- (1) The method relies less on external guidance information and does not require precise mathematical models of the environment and tasks
- (2) The possibility of building a library of actions based on existing tactical rules, which can effectively use existing experience and improve the efficiency of task assignment.
- (3) The method can be applied to the task allocation problem under the dynamic change strip of combat resources and target tasks.

However, the following challenges are faced when deep reinforcement learning methods are used to solve the task assignment problem. First, when there are complex coupling relationships between tasks, the convergence speed of reinforcement learning can be slow, and how to narrow the exploration space of reinforcement learning through experience is a problem worth investigating. Second, most of the existing studies are conducted under ideal communication conditions, i.e., all the intelligence can have all the state information of the environment at the same time,

which is different from practical applications, and how to carry out the task assignment method under unstable communication conditions is also a topic worthy of research.

4.3. Military Chess Deduction

4.3.1. Mission Characteristics Analysis. Military chess is a collective term for the simulation of military operations of two or more opposing sides using rules, data, and phases to describe actual or hypothetical situations, and it is an essential tool for analyzing warfare. The computerized chess platform can simulate the military actions of two or more opposing sides in a randomized manner, providing a decision context and test environment for studying intelligent military decision-making that closely resembles real warfare. It is a typical multi-intelligence decision-making environment where the commander (chess player) controls multiple combat entities (pawns) of the same or different types, enabling them to work together to achieve the overall operational objectives, which makes chess rehearsal highly similar to strategy-based video games. The action-space complexity of this problem grows exponentially as the autonomy and size of the intelligence increase, making the large-scale chess projection problem extremely challenging.

4.3.2. Advantages and Disadvantages of Existing Methods. Traditional pawn projection methods are mainly performed using knowledge-based reasoning-type decision-making methods based on expert knowledge. The knowledge-based inference-based decision-making method is to form domain knowledge by manually extracting unstructured information such as combat rules, ordinance regulations, and commander's experience and reorganizing it into a formatted expert experience on which decisions are made. This method has the following characteristics. First, once the expert experience base is established, it can be directly used for military chess rehearsal, which can provide intuitive and immediate verification of expert experience. Second, the quality of decision-making will not be too low. Most of the artificially extracted knowledge originates from the rules of experience accumulated by the commanders over many years and is a tactical strategy that can reflect the art of command to a certain extent, so the decision results of this method can conform to military common sense without producing excessively poor decision results. However, there are two obvious problems with the knowledge-based reasoning decision-making method. First, it is difficult to accurately describe the expert knowledge provided by the military commander using a formatted computer language. Second, knowledge-based reasoning decision-making methods are limited by the level of human intelligence and exhibit behavioral actions that are usually fixed, thus making them vulnerable to adversaries' shortcomings.

4.3.3. Advantages and Challenges of Deep Reinforcement Learning Methods. Deep reinforcement learning has already achieved excellent performance in strategy-based video

games such as StarCraft and King's Quest, which have shown the great potential of deep reinforcement learning methods in pawn pushing. Compared to knowledge-based decision-making methods, deep reinforcement learning has the following advantages over knowledge-based decision-making methods.

- (1) A high degree of automation. Algorithms can automatically learn from large amounts of data without manually refining operational planning and describing it in computer language.
- (2) The high upper limit of decision quality. Deep reinforcement learning-based decision-making in military chess has the potential to surpass the decision-making capabilities of human military commanders because of its high flexibility in acquiring knowledge from large amounts of data that humans do not possess.

However, the following challenges are also faced when applying deep reinforcement learning methods to the task of soldier chess deduction.

- (1) Poor model interpretation and unstable generalization performance. Decision neural network models are obtained by deep reinforcement learning, and the decision rules are hidden in the structure and weights of the network, and these decision rules are usually difficult to be expressed explicitly. And when the training scenario differs significantly from the application scenario, the neural network is usually often not guaranteed to provide similar quality decision results.
- (2) High-quality warfare strategies are difficult to learn. Deep reinforcement learning yields mostly the underlying action-level model, i.e., the state-action response output model. For more complex decision scenarios, the search space for model learning training is extremely large, and it is often difficult to learn high-quality battle strategies in a limited time.

5. Conclusion

This paper outlines the development of deep reinforcement learning algorithms and their research results in relevant application scenarios and points out the problems and directions of development. In general, the success of various DRL methods is mainly due to the substantially increased computational power, but compared with human intelligence, DRL methods still have problems such as low learning efficiency and single application scenarios. In the future, DRL will develop in the following directions: (1) developing more complete and efficient computational graph models to facilitate the addition of attention mechanisms, memory units, feedback control, and other auxiliary structures to DRL networks; (2) integrating different kinds of memory units in DRL models, such as LSTM, memory stack memory, NTM, and other models, so that the memory function of agents will be more perfect (3) to integrate different kinds of memory units in the DRL model, such as

LSTM, memory stack memory, NTM, and other models, so that the memory function of the agent can be improved and its active reasoning and cognitive ability can be enhanced; (4) to improve the driveability of the DRL model by using transfer learning and other methods; (5) to further strengthen the inspiration of cognitive neuroscience to DRL, so that the agent can gradually master the functions of memory, focus, planning and learning as the human brain.

DRL has initially shown a broad application prospect in military intelligence. Air warfare decision-making, mission assignment, and military chess deduction are typical military application scenarios of reinforcement learning. In terms of military applications of DRL models, a more realistic and accurate environment description model is needed in the future, enabling deep reinforcement learning to play a more critical role in military intelligence.

Data Availability

The data used to support the findings of this article are available from the corresponding author upon request.

Conflicts of Interest

The authors declare that they have no conflicts of interest.

Acknowledgments

This work has been funded by the National Natural Science Foundation of China, grant no. 61703427.

References

- [1] O. Vinyals, I. Babuschkin, W. M. Czarnecki et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, 2019.
- [2] S. Grigorescu, B. Trasnea, T. Cocias, and G. Macesanu, "A survey of deep learning techniques for autonomous driving," *Journal of Field Robotics*, vol. 37, no. 3, pp. 362–386, 2020.
- [3] M. Yuan, J. Shan, and K. Mi, "Deep reinforcement learning based game-theoretic decision-making for autonomous vehicles," *IEEE Robotics and Automation Letters*, vol. 7, pp. 818–825, 2022.
- [4] O. Perez-Gil, R. Barea, E. Lopez-Guillen et al., "Deep reinforcement learning based control for Autonomous Vehicles in CARLA," *Multimedia Tools and Applications*, vol. 81, pp. 3553–3576, 2022.
- [5] Y. Dai, D. Xu, S. Maharjan, Z. Chen, Q. He, and Y. Zhang, "Blockchain and deep reinforcement learning empowered intelligent 5G beyond," *Ieee Network*, vol. 33, pp. 10–17, 2019.
- [6] X. He, K. Wang, H. Huang, T. Miyazaki, Y. Wang, and S. Guo, "Green resource allocation based on deep reinforcement learning in content-centric IoT," *Ieee Transactions on Emerging Topics in Computing*, vol. 8, pp. 781–796, 2020.
- [7] C. H. Liu, Q. Lin, and S. Wen, "Blockchain-enabled data collection and sharing for industrial IoT with deep reinforcement learning," *IEEE Transactions on Industrial Informatics*, vol. 15, pp. 3516–3526, 2019.
- [8] M. Liu, F. R. Yu, Y. Teng, V. C. M. Leung, and M. Song, "Performance optimization for blockchain-enabled industrial internet of things (IIoT) systems: a deep reinforcement learning approach," *IEEE Transactions on Industrial Informatics*, vol. 15, pp. 3559–3570, 2019.
- [9] H.-n. Wang, N. Liu, Y.-y. Zhang et al., "Deep reinforcement learning: a survey," *Frontiers of Information Technology and Electronic Engineering*, vol. 21, pp. 1726–1744, 2020.
- [10] N. Vithayathil Varghese and Q. H. Mahmoud, "A survey of multi-task deep reinforcement learning," *Electronics*, vol. 9, 2020.
- [11] D. Cao, W. Hu, J. Zhao et al., "Reinforcement learning and its applications in modern power and energy systems: a review," *Journal of Modern Power Systems and Clean Energy*, vol. 8, pp. 1029–1042, 2020.
- [12] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "Deep reinforcement learning A brief survey," *IEEE Signal Processing Magazine*, vol. 34, pp. 26–38, 2017.
- [13] V. Mnih, K. Kavukcuoglu, D. Silver et al., "Playing Atari with deep reinforcement learning," 2013, <https://arxiv.org/abs/1312.5602>.
- [14] V. Mnih, K. Kavukcuoglu, D. Silver et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.
- [15] Z. Wang, N. de Freitas, and M. Lanctot, "Dueling network architectures for deep reinforcement learning," in *Proceedings of the 33rd International Conference on International Conference on Machine Learning*, New York NY USA, June 2015.
- [16] M. J. Hausknecht and P. Stone, "Deep recurrent q-learning for partially observable mdps," 2015, <https://arxiv.org/abs/1507.06527>.
- [17] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double q-learning," in *Proceedings of the 13th AAAI Conference on Artificial Intelligence*, Phoenix AZ, USA, February 2015.
- [18] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," 2016, <https://arxiv.org/abs/1511.05952>.
- [19] S. Gu, T. P. Lillicrap, I. Sutskever, and S. Levine, "Continuous deep q-learning with model-based acceleration," *CoRR*, in *Proceedings of the 33rd International Conference on International Conference on Machine Learning*, New York NY USA, June 2016.
- [20] M. Fortunato, M. G. Azar, B. Piot et al., "Noisy networks for exploration," in *Proceedings of the ICLR 2018 Conference Blind Submission*, Vancouver, BC, Canada, April 2018.
- [21] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proceedings of the 34th International Conference on Machine Learning*, Sydney NSW, Australia, August, 2017.
- [22] T. P. Lillicrap, J. J. Hunt, A. Pritzel et al., "Continuous control with deep reinforcement learning," 2019, <https://arxiv.org/abs/1509.02971>.
- [23] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, "Trust region policy optimization," 2017, <https://arxiv.org/abs/1502.05477>.
- [24] V. Mnih, A. P. Badia, M. Mirza et al., "Asynchronous methods for deep reinforcement learning," 2016, <https://arxiv.org/abs/1602.01783>.
- [25] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy Gradient Methods for Reinforcement Learning with Function Approximation," in *Proceedings of the 12th International Conference on Neural Information Processing Systems*, Denver, CO, USA, November 1999.
- [26] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic Policy Gradient Algorithms," in *Proceedings of the 31st International Conference on*

- International Conference on Machine Learning*, Beijing China, June 2014.
- [27] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," 2018, <https://arxiv.org/abs/1802.09477>.
 - [28] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, <https://arxiv.org/abs/1707.06347>.
 - [29] T. Haarnoja, A. Zhou, K. Hartikainen et al., "Soft actor-critic algorithms and applications," 2019, <https://arxiv.org/abs/1812.05905>.
 - [30] A. Mohammadzadeh, M. H. Sabzalian, and W. Zhang, "An interval type-3 fuzzy system and a new online fractional-order learning algorithm: theory and practice," *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 9, pp. 1940–1950, 2020.
 - [31] K. Zhang, Z. Yang, and T. Basar, "Decentralized multi-agent reinforcement learning with networked agents: recent advances," *Frontiers of Information Technology and Electronic Engineering*, vol. 22, pp. 802–814, 2021.
 - [32] H. Wang, K. A. Alattas, A. Mohammadzadeh, M. H. Sabzalian, A. A. Aly, and A. Mosavi, "Comprehensive review of load forecasting with emphasis on intelligent computing approaches," *Energy Reports*, vol. 8, Article ID 13189, 2022.
 - [33] M. Hessel, J. Modayil, H. V. Hasselt et al., "Rainbow: combining improvements in," *deep reinforcement learning*, 2017.
 - [34] F. Li, X. Fuhuai, M. Guanglei, and W. Dongzheng, "An uav air-combat decision expert system based on receding horizon control," *Journal of Beijing University of Aeronautics and Astronautics*, vol. 41, pp. 1994–1999, 2015.
 - [35] Y. Zhang and D. Luo, "Editorial of special issue on UAV autonomous, intelligent and safe control," *Guidance, Navigation and Control*, vol. 1, no. 4, 2021.
 - [36] J. Hu, L. Wang, T. Hu, C. Guo, and Y. Wang, "Autonomous maneuver decision making of dual-UAV cooperative air combat based on deep reinforcement learning," *Electronics*, vol. 11, 2022.
 - [37] D. Hu, R. Yang, J. Zuo, Z. Zhang, J. Wu, and Y. Wang, "Application of deep reinforcement learning in maneuver planning of beyond-visual-range air combat," *IEEE Access*, vol. 9, pp. 32282–32297, 2021.
 - [38] J. B. Rawlings and K. R. Muske, "The stability of constrained receding horizon control," *IEEE Transactions on Automatic Control*, vol. 38, no. 10, pp. 1512–1516, 1993.
 - [39] H. Michalska and D. Q. Mayne, "Receding horizon control of nonlinear systems," in *Proceedings of the 28th IEEE Conference on Decision and Control*, Tampa, FL, USA, December, 1989.
 - [40] A. L. Leatham and U. Linch, "Two Numerical Methods to Solve Realistic Air-To-Air Combat Differential Games," in *Proceedings of the 12th Aerospace Sciences Meeting*, Washington, DC, USA, January, 1974.
 - [41] A. Ardema, M. Heymann, and N. Rajan, "Analysis of a combat problem: the turret game," *Journal of Optimization Theory and Applications*, vol. 54, no. 1, pp. 23–42, 1987.
 - [42] H. Park, B.-Y. Lee, M.-J. Tahk, and D.-W. Yoo, "Differential game based air combat maneuver generation using scoring function matrix," *International Journal of Aeronautical and Space Sciences*, vol. 17, no. 2, pp. 204–213, 2016.
 - [43] Q. Li, Y. Chen, Z. Huang, Y. Sun, and C. Feng, "An algorithm of air combat maneuver strategy based on two layer game decision-making and distributed mcts method with double game trees under uncertain interval information conditions," in *Proceedings of the 33rd Chinese Control and Decision Conference (CCDC)*, pp. 6875–6880, Kunming, China, May 2021.
 - [44] Y. Ma, X. Ma, and X. Song, "A case study on air combat decision using approximated dynamic programming," *Mathematical Problems in Engineering*, vol. 2014, Article ID 183401, 10 pages, 2014.
 - [45] W. Lin, "Decision-making for Air Combat Maneuvering Based on Hybrid Algorithm," *Systems engineering and electronics*, vol. 35, 2013.
 - [46] A. S. Tasbas and S. U. Aydinli, "2-d air combat maneuver decision using reinforcement learning," in *Proceedings of the 2021 International Conference on Engineering and Emerging Technologies (ICEET)*, pp. 1–6, Istanbul, Turkey, October, 2021.
 - [47] X. Zhang, G. Liu, C. Yang, and J. Wu, "Research on Air Combat Maneuver Decision-Making Method Based on Reinforcement Learning," *Electronics*, vol. 7, 2018.
 - [48] L. Zhong, L. Zhao, C. Ding et al., "Vision-based 3d aerial target detection and tracking for maneuver decision in close-range air combat," *IEEE Access*, vol. 10, pp. 4157–4168, 2022.
 - [49] L. Bo, L. Shiyang, C. Daqing, and L. Xitong, "A decision-making method for air combat maneuver based on hybrid deep learning network," *Chinese Journal of Electronics*, vol. 31, no. 1–9, 2022.